WOLF

PROGRAMMING LANGUAGE

Builder PRD · v4.0 · The Real Build

Natively compiled · LLVM backend · PHP familiarity · Faster than Go

Version	Date	Status
4.0 — The Real Build	February 2026	ACTIVE

Team: Human Founder + Claude Opus 4.6 + Gemini 3

How To Use [redacted]cument

> *This is the builder's manual for the Wolf programming language. Every section answers: what to build, how to build it, and how to know when it's done. This PRD was built by reading a real production PHP codebase (Selledge API) and translating every pattern into Wolf. Nothing here is invented — it is all grounded in code that works in production today.*

This PRD has five parts:

- Part A — Vision & Architecture: What Wolf is, who it's for, and every locked technical decision.
- Part B — Language Specification: Every syntax rule, library pattern, routing system, and database driver.
- Part C — Sprint Task Boards: 12 weeks of tasks with owners and acceptance criteria.
- Part D — Code Specification: Exact Go structs and interfaces for Claude and Gemini to implement.
- Part E — Testing & Launch: How to verify Wolf works and how to ship it.

Major changes from v3.1: (1) Go transpilation removed — Wolf compiles natively via LLVM. (2) No :: anywhere in application code — banned permanently. (3) No DB:: facade — database is always new Database / new Redis in libraries. (4) Routing is URL-pattern based like TraversyMVC Core.php — auto-discovered, not manually wired. (5) Plugin architecture for optional libraries — Cache, JobQueue are developer-built, not core. (6) wolf generate command added. (7) Full backend vision documented — Phase 2 is frontend via WebAssembly. (8) European/legacy PHP market positioning added.

PART A — VISION & ARCHITECTURE

A1. The Wolf Vision

A1.1 What Wolf Is — One Paragraph

Wolf is a natively compiled, general-purpose programming language that feels like PHP, performs faster than Go, and runs on every major platform as a single binary with no runtime dependencies. Wolf compiles to native machine code via LLVM — the same backend that powers Rust and Swift. Wolf developers write code that looks familiar from day one, ship production APIs in hours, and never fight a compiler or configure a web server. The database pattern is TraversyMVC. The routing is automatic. The performance is native.

A1.2 Who Wolf Is For

Primary target	PHP developers who want performance without learning a new paradigm. They know $variables, $this->db->, and controller/method/param routing. Wolf gives them all of that plus LLVM speed.
Secondary target	Backend developers from any language who want a fast, easy-to-read language for APIs, data pipelines, and ML-integrated services.
Enterprise target	German, French, Dutch, Polish, and Brazilian companies running legacy PHP systems who need a modern language their existing PHP teams can adopt in days, not months.
Long-term target	Any developer building full-stack applications — backend today, frontend via WebAssembly in Phase 2.

A1.3 The Market Opportunity

PHP powers approximately 77% of all websites with a known server-side language. Germany alone has thousands of Mittelstand companies running business-critical systems on PHP codebases that are 10–20 years old. These companies cannot modernise to Go or Rust without retraining their entire engineering teams. Wolf is the bridge — PHP familiarity, modern performance, no retraining cost.

The languages Wolf competes with and why Wolf wins:

Language	What it does well	What Wolf does better
PHP	Familiar syntax, huge ecosystem	Native compilation — 10–50x faster. No Apache required. Single binary deployment.
Go	Fast, simple, great for APIs	Wolf is faster (no GC pauses). Wolf syntax is more familiar to the 77% who know PHP.
Python	ML ecosystem, easy to learn	Wolf has the same ML access via @ml

		blocks, plus native speed, plus PHP-familiar syntax.
Node.js/TypeScript	Frontend devs doing backend	Wolf is natively compiled and faster. Wolf syntax is simpler for pure backend work.
Rust	Maximum performance	Wolf is nearly as fast (LLVM backend) but infinitely more accessible. No borrow checker.

A1.4 The Five Locked Pillars

1. PHP familiarity	Dollar-sign variables. $this->method() syntax. Curly brace blocks. No indentation rules. No imports for stdlib. A PHP developer reads Wolf without a manual.
2. Native compilation	Wolf compiles to native machine code via LLVM. One binary, no runtime, no dependencies. wolf build produces a file you scp to a server and run. Done.
3. Faster than Go for full backend	LLVM optimisation + no garbage collector = no GC pauses. Fixed thread pool + async I/O for web workloads. Benchmarks will show Wolf ahead of Go for API throughput and latency consistency.
4. TraversyMVC database pattern	Database connection class in libraries/. Constants in config/config.wolf. Models use new Database / new Redis in constructor. $this->db->query/bind/execute/single/resultSet. No DB:: facade anywhere. No :: in application code anywhere.
5. Zero-config routing	Controllers auto-discovered from controllers/ directory. URL format /controller/method/param — exactly like Core.php. No route files. No manual wiring. Drop a controller file, routes exist.

A1.5 What Wolf Is NOT

- NOT a transpiler — Wolf does not produce Go, JavaScript, or any other language as output.
- NOT Go-dependent — the user never installs Go. Wolf's compiler is written in Go but ships as a self-contained binary.
- NOT Python-dependent for non-ML code — Python is only needed when @ml blocks are used. wolf.python manages the venv automatically.
- NOT framework-first — Wolf is a language. WolfMVC is a Phase 2 framework built on top of Wolf.
- NOT Apache-dependent — Wolf's compiled server handles HTTP natively. No .htaccess, no Nginx config, no web server required.
- NOT :: in application code — this syntax is permanently banned from any code a developer writes.

A2. Architect[redacted]he Comp[redacted]

A2.1 The Bootstrap Reality

Before any of the language features matter, the compiler must exist. The Wolf compiler is written in Go and ships as a single native binary. This is the same approach used by Go itself — Go's compiler is written in Go, but users just download go and run it. Wolf users download wolf and run it. They never know or care what the compiler is written in.

> *The Wolf compiler being written in Go does not make Wolf a Go transpiler. It makes Go the implementation language of the Wolf compiler — exactly like V8 being written in C++ does not make JavaScript a C++ transpiler.*

A2.2 Compiler Pipeline — 8 Stages

#	Stage	Input → Output	Responsibility
1	**Lexer**	`Source → []Token`	Tokenise Wolf source. $vars, @ml, string interpolation, # comments. Typed tokens with line+col.
2	**Parser**	`[]Token → AST`	Recursive-descent parser. Typed AST nodes. Clear errors with line/col and helpful message.
3	**Resolver**	`AST → Ann. AST`	Resolve variable scopes. Strip $ for LLVM output. Detect undeclared variables.
4	**Type Checker**	`Ann. AST → Typed AST`	Dynamic mode by default. Strict mode on strict keyword. Always validate @ml boundaries.
5	**IR Emitter**	`Typed AST → WIR`	Wolf Intermediate Representation. Flatten closures, normalise interpolation, resolve constants.
6	**LLVM IR Emitter**	`WIR → .ll file`	Emit LLVM IR. @ml → C bridge calls. async → thread pool tasks. Database class → driver calls.
7	**LLVM Build**	`ll file → native binary`	llc + clang link. Single native binary. No external runtime needed. User never sees .ll file.
8	**wolf run**	`Source → execute`	Stages 1–7 to temp dir, runs binary. For development. wolf build for production.

A2.3 LLVM — Why It Beats Go Transpilation

No Go dependency	User installs Wolf once. No Go, no GOPATH, no go.sum. Single binary download and run.
No Go errors	Wolf compiler errors are Wolf errors. The user never sees a Go type error from generated code.
Native performance	LLVM applies the same optimisations as Rust and Swift. Output is near-C speed.
No GC pauses	Wolf runtime has no garbage collector. No latency spikes under load. Consistent API response times.

Cross-platform free	LLVM targets Linux, macOS, Windows, ARM, WASM. Wolf gets all platforms from one emitter.
Future WASM	LLVM's WASM backend gives Wolf frontend compilation (Phase 2) almost for free.

A2.4 Technology Stack

Compiler language	Go 1.22+. Compiler is a Go module. Ships as a self-contained binary.
Compiler output	LLVM IR (.ll files). LLVM compiles to native binary for target platform.
LLVM binding	llir/llvm Go package. Programmatic IR construction — no string templating.
C bridge	wolf_runtime.c — wraps CPython C API for @ml blocks. Linked at compile time.
Python env	wolf.python config + auto-managed venv. Developer never runs pip or python directly.
HTTP server	Wolf's own HTTP server in the Wolf runtime — no Nginx, no Apache, no .htaccess.
MySQL driver	Go mysql driver compiled into Wolf runtime. Available as new Database.
Redis driver	Go redis driver compiled into Wolf runtime. Available as new Redis.
MongoDB driver	Go mongo driver compiled into Wolf runtime. Available as new Mongo.
PostgreSQL driver	Go postgres driver compiled into Wolf runtime. Available as new Database (pg driver).
CLI framework	cobra for wolf subcommands (build, run, new, generate, fmt, test, python).
CI/CD	GitHub Actions. Every PR: tests + build all platforms. Merge to main: publish release binaries.

A2.5 Project Structure — Every Wolf App

```
myapp/  config/    config.wolf          ← DB constants + app constants. Reads from .env.
libraries/             ← framework core (wolf new generates these)    Core.wolf
← router, boot, serve — the engine    Controller.wolf      ← base class all controllers
extend    Database.wolf        ← MySQL/PostgreSQL connection class  models/
← data models. All use new Database in constructor.  controllers/           ← all extend
Controller. Auto-discovered by Core.  services/              ← business logic.
Instantiated in controllers.  helpers/              ← project-specific helper functions
public/    assets/              ← static files (images, CSS, JS)  wolf.python
← Python requirements (only needed for @ml blocks)  .env                   ← real
credentials — NEVER committed to git  .env.example           ← key template — committed
to git  .gitignore             ← includes .env, wolf_out/, .wolf-python/  server.wolf
← 3 lines. Written once by wolf new. Never touched again.  wolf.mod              ←
module definition  README.md# Optional libraries — developer creates only if needed:
libraries/    Cache.wolf           ← wolf generate library Cache    JobQueue.wolf
```

```
← wolf generate library JobQueue    Mailer.wolf          ← wolf generate library Mailer
Anything.wolf        ← whatever the project needs
```

PART B — LANGUAGE SPECIFICATION

B1. Syntax Rules — The Non-Negotiables

> *These rules define Wolf's feel. They do not change. Every feature built must comply with all of them.*

No :: anywhere	The double-colon operator does not exist in Wolf application code. Ever. Static calls use instances. Stdlib functions are global.
No :: in libraries	Even inside library classes (Core, Database, Controller), :: is never used. Method calls always go through instances.
Dollar variables always	Every variable is $name. No exceptions. Functions use $param. Classes use $this->property.
Curly brace blocks	All blocks use {}. No indentation rules. Whitespace is style, not syntax.
No imports for stdlib	json_encode(), password_hash(), jwt_encode(), mail_send() — all available everywhere. No require, no use, no import.
No require/include	Wolf's compiler auto-discovers all classes at compile time. No manual loading.
$this->db->	Database access in models is always $this->db->query/bind/execute. No exceptions.
new ClassName	Instantiation is always new ClassName. No factory methods, no static constructors.
Dynamic by default	Variables hold any type. Strict mode is opt-in per file with strict keyword at top.
Server is built in	wolf build produces a binary that serves HTTP. No Apache, Nginx, or .htaccess.

B2. Variables, Types & Operators

```
# Dynamic (default)$name    = "Wolf"$count   = 0$price   = 9.99$active  = true$items   =
[1, 2, 3]$user    = {"id": 1, "name": "Ade"}$nothing = nil# Strict (opt-in per file)
strictvar $userId:  string  = "HiveShareJan.01.2026..."var $amount:  float   = 9.99var
$active:  bool    = truevar $scores:  []float = [9.1, 8.7]# String interpolation$city  =
"Lagos"$msg   = "Pickup in {$city}"$total = "Total: ${$price * $qty} NGN"$info  =
"Driver {$driver->name} is {$driver->rating} stars"# Operators — identical to PHP$a + $b
$a - $b   $a * $b   $a / $b   $a % $b$a == $b  $a != $b  $a < $b   $a > $b   $a <= $b
$a >= $b$a && $b  $a || $b  !$a$a++  $a--  $a += $b  $a -= $b  $a .= $b  # string concat
$a ?? $b  # null coalescing — if $a is nil, use $b
```

B3. Function[redacted]ses

```
# Basic functionfunc greet($name) {    return "Hello {$name}"}# Multiple return values
func divide($a, $b) -> (float, error) {    if $b == 0 { return 0, error("division by
zero") }    return $a / $b, nil}# Arrow shorthandfunc double($n) => $n * 2# Default
parametersfunc createUser($name, $role = "passenger") { ... }# Named parametersfunc
connect($host, $port = 3306, $timeout = 30) { ... }connect(host: "localhost", timeout:
60)# Class — identical feel to PHPclass Driver {    private $name    private $rating
public  $online = false    func __construct($name, $rating) {        $this->name   =
$name        $this->rating = $rating    }    func goOnline()  { $this->online = true }
func summary() => "{$this->name} — {$this->rating}★"}$driver = new Driver("Ade", 4.8)
$driver->goOnline()print($driver->summary())  # Ade — 4.8★# Inheritanceclass
EliteDriver extends Driver {    func __construct($name)
{        parent::__construct($name, 5.0)  # parent:: is the ONE allowed :: — inside
classes only    }}
```

B4. Control Flow

```
# if / else if / elseif $score > 90      { $grade = "A" }else if $score > 75 { $grade =
"B" }else                { $grade = "C" }# for — C-stylefor $i = 0; $i < 10; $i++
{ print($i) }# foreach — identical to PHPforeach $items as $item
{ print($item) }foreach $users as $id => $user    { print("{$id}: {$user->name}") }#
whilewhile $running { $running = checkStatus() }# match — exhaustive switch, compiles to
LLVM switch instructionmatch $status {    "active"    => handleActive()    "pending"
=> handlePending()    "cancelled" => handleCancel()    _           => handleDefault()}#
try / catch — error handlingtry {    $user = fetchUser($id)} catch ($e) {    return
$this->res->error($e->message, 500)}# Multiple return values — Go-style[$data, $err] =
parseInput($raw)if $err != nil { return $this->res->error($err, 400) }
```

B5. Concurrency

```
# async — runs in Wolf's thread pool, non-blocking$task = async { return
fetchDrivers($location) }$drivers = await $task# parallel — run multiple tasks, wait for
ALLparallel {    $drivers  = fetchNearbyDrivers($zone)    $surge    =
getSurgeMultiplier($zone)    $eta      = estimateArrival($pickup, $dropoff)}# all three
complete before this line runs# channels$ch = channel(int)async { send($ch,
computeResult()) }$value = receive($ch)
```

B6. Routing System — How Wolf Serves Requests

> *This section is the direct Wolf translation of Core.php from the Selledge codebase. The URL pattern, camelCase conversion, default controller/method, and param extraction all work identically. The difference is Wolf compiles this at build time — zero runtime overhead.*

B6.1 URL Pattern — Identical to TraversyMVC

Wolf uses the exact same URL format as TraversyMVC's Core.php:

```
# URL format:/controller/method/param1/param2/param3# Examples:GET  /users/findById/42
→ UsersController::findById(42)POST /rides/create                →
RidesController::create()GET  /auth/login                  → AuthController::login()GET
/listings/getByCategory/electronics/page/2                                  →
ListingsController::getByCategory('electronics', 'page', '2')# Default (no controller in
URL):GET  /                            → HomeController::index()# camelCase conversion —
identical to Core.php:GET  /users/find-by-email/ada@wolf.dev →
UsersController::findByEmail('ada@wolf.dev')
```

B6.2 server.wolf — Written Once, Never Touched Again

wolf new generates this file. The developer never opens it again. Adding new endpoints means adding controller methods — nothing else.

```
// server.wolf// Generated by wolf new. Do not edit.$core = new Core
$core->cors([    "http://localhost:3000",    "http://localhost:3001",
"https://app.myapp.com"])$core->boot()$core->serve(8080)
```

B6.3 libraries/Core.wolf — The Engine

Mirrors Core.php exactly. Auto-discovers controllers. Parses URLs. Dispatches to methods. Handles camelCase conversion. Manages middleware.

```
// libraries/Core.wolf// Wolf's equivalent of Core.php — the router and boot engine.
class Core {    private $router    private $corsOrigins = []    func __construct()
{        $this->router = new Router    }    func cors($origins)
{        $this->corsOrigins = $origins    }    func boot() {        # Auto-discover all
*Controller.wolf files        $files = scan_dir(app_root() . "/controllers/")
foreach $files as $file
{            $this->router->register(class_from_file($file))        }    }    func
serve($port = 8080) {        log("   Wolf running on port {$port}")
$this->router->listen($port, $this->corsOrigins)    }}
```

B6.4 libraries/Controller.wolf — Base Class

Every controller extends this. Provides $this->req and $this->res automatically. Matches the base Controller.php pattern.

```
// libraries/Controller.wolfclass Controller {    protected $req    protected $res
func __construct($req, $res) {        $this->req = $req        $this->res = $res    }}
```

B6.5 A Real Controller — How Developers Write Them

```
// controllers/AuthController.wolfclass AuthController extends Controller {    # POST
/auth/login    func login() {        $data = $this->req->body()        $user = new User
$found = $user->findUserByEmailOrPhone($data["username"])        if !$found
{            return $this->res->json(["status" => false, "message" => "Invalid
credentials"], 401)        }        if !password_verify($data["password"],
$found->password_hash) {            return $this->res->json(["status" => false,
```

```
"message" => "Invalid credentials"], 401)        }        $token =
jwt_encode([            "user_id" => $found->user_id,            "email"   =>
$found->email,        ], JWT_SECRET)        return $this->res->json(["status" => true,
"token" => $token])    }    # POST /auth/register    func register() {        $data =
$this->req->body()        $user = new User        if
$user->findUserByEmail($data["email"]) {            return $this->res->json(["status" =>
false, "message" => "Email already registered"], 409)        }        $data["password"]
= password_hash($data["password"])        $userId = $user->register($data)        return
$this->res->json(["status" => true, "user_id" => $userId], 201)    }}
```

B7. Database — The TraversyMVC Pattern

> *The database pattern is locked and final. It mirrors the Selledge production codebase exactly. No DB:: facade. No :: in application code. Always new Database / new Redis in the constructor. Always $this->db-> to query.*

B7.1 config/config.wolf

```
// config/config.wolf// Constants only. Reads secrets from .env.// This file is
committed to git. Secrets never are.# Database — MySQLdefine("DB_HOST",
Env::get("DB_HOST", "localhost"))define("DB_USER",   Env::get("DB_USER", "root"))
define("DB_PASS",   Env::get("DB_PASS", ""))define("DB_NAME",   Env::get("DB_NAME",
"myapp"))define("DB_PORT",   Env::get("DB_PORT", "3306"))# Redisdefine("REDIS_HOST",
Env::get("REDIS_HOST", "localhost"))define("REDIS_PORT", Env::get("REDIS_PORT", "6379"))
define("REDIS_PASS", Env::get("REDIS_PASS", ""))# MongoDBdefine("MONGO_HOST",
Env::get("MONGO_HOST", "localhost"))define("MONGO_PORT", Env::get("MONGO_PORT",
"27017"))define("MONGO_DB",   Env::get("MONGO_DB",   "myapp"))# Application
define("APP_NAME",      Env::get("APP_NAME",      "MyApp"))define("APP_VERSION",
"1.0.0")define("APPROOT",       app_root())define("URLROOT",       Env::get("URLROOT",
"http://localhost:8080"))define("SITENAME",      Env::get("SITENAME",      "MyApp"))
define("APP_REQUEST_ID",Env::get("APP_REQUEST_ID","rid_0000"))# Authdefine("JWT_SECRET",
Env::get("JWT_SECRET",    "change-me-in-env"))define("JWT_EXPIRE",
Env::get("JWT_EXPIRE",    "3600"))
```

B7.2 libraries/Database.wolf — MySQL/PostgreSQL

```
// libraries/Database.wolf// Wolf's PDO equivalent. Mirrors the Selledge Database class
exactly.// Generated by wolf new. Developer rarely edits this.class Database
{    private $host   = DB_HOST    private $user   = DB_USER    private $pass   = DB_PASS
private $dbname = DB_NAME    private $port   = DB_PORT    private $dbh    # connection
handle    private $stmt   # prepared statement    private $error    func __construct()
{        $dsn =
"mysql:host={$this->host};port={$this->port};dbname={$this->dbname};charset=utf8mb4"
try {            $this->dbh = db_connect($dsn, $this->user, $this->pass,
[                "persistent" => true,                "errmode"    =>
"exception",            ])        } catch ($e) {            $this->error = $e->message
log("DB Error: {$this->error}")        }    }    func query($sql)
{ $this->stmt = $this->dbh->prepare($sql) }    func bind($param, $value, $type = nil)
{        if $type == nil {            match true {                is_int($value)   =>
$type = "int"                is_bool($value)  => $type = "bool"
is_null($value)  => $type = "null"                _                => $type =
"string"            }        }        $this->stmt->bindValue($param, $value, $type)    }
```

```
func execute()      { return $this->stmt->execute() }    func resultSet()
{ $this->execute(); return $this->stmt->fetchAll() }    func single()
{ $this->execute(); return $this->stmt->fetch() }    func rowCount()     { return
$this->stmt->rowCount() }    func lastInsertId() { return $this->dbh->lastInsertId() }
func beginTransaction() { return $this->dbh->beginTransaction() }    func commit()
{ return $this->dbh->commit() }    func rollBack()         { return
$this->dbh->rollBack() }}
```

B7.3 Built-in Database Drivers — new ClassName, Zero Config

Redis, MongoDB, and PostgreSQL are built-in drivers. The developer instantiates them exactly like Database. No files to create. No libraries to generate. Just configure the constants and use them.

```
# Redis — built-in, reads REDIS_HOST/PORT/PASS from configclass SessionModel
{    private $redis    func __construct() { $this->redis = new Redis }    func
setSession($userId, $data, $ttl = 3600) {        $this->redis->set("session:{$userId}",
json_encode($data), $ttl)    }    func getSession($userId) {        return
json_decode($this->redis->get("session:{$userId}"))    }    func destroySession($userId)
{        $this->redis->del("session:{$userId}")    }}# MongoDB — built-in, reads
MONGO_HOST/PORT/DB from configclass EventModel {    private $mongo    func __construct()
{ $this->mongo = new Mongo }    func logEvent($data)
{        $this->mongo->collection("events")->insertOne($data)    }    func
getEvents($filter) {        return
$this->mongo->collection("events")->find($filter)    }}# PostgreSQL — same Database
class, different DSN in config# Set DB_DRIVER=postgres in .env# Database class detects
driver and uses correct DSN automatically
```

B7.4 A Real Model — Using $this->db->

This is a direct Wolf translation of the Selledge User model. Every method works identically.

```
// models/User.wolfclass User {    private $db    func __construct() {        $this->db
= new Database    }    func generateCustomUserId() {        return "HiveShare" .
date("M.d.Y") . time() . date("H.s") . md5(date("ymdhms") . rand(1000, 9999))    }
func register($data) {        $userId = $this->generateCustomUserId()
$this->db->query("INSERT INTO initkey_rid                        (user_id, first_name,
last_name, email, phone, password_hash)                        VALUES
(:user_id, :first_name, :last_name, :email, :phone, :password_hash)")
$this->db->bind(":user_id",       $userId)        $this->db->bind(":first_name",
$data["first_name"])        $this->db->bind(":last_name",     $data["last_name"])
$this->db->bind(":email",         $data["email"])        $this->db->bind(":phone",
$data["phone"])        $this->db->bind(":password_hash", $data["password"])        if
$this->db->execute() {            $this->assignRole($userId, "passenger")
$this->createWallet($userId)            return $userId        }        return false    }
func findUserByEmailOrPhone($username) {        $this->db->query("SELECT * FROM
initkey_rid WHERE email = :u OR phone = :u")        $this->db->bind(":u", $username)
return $this->db->single()    }    func findUserByEmail($email)
{        $this->db->query("SELECT id FROM initkey_rid WHERE email = :email")
$this->db->bind(":email", $email)        $this->db->single()        return
$this->db->rowCount() > 0    }    func findUserByEmailAndGetDetails($email)
{        $this->db->query("SELECT u.*, w.balance, w.held_funds, w.currency,
w.status as wallet_status                        FROM initkey_rid u
LEFT JOIN wallets w ON u.user_id = w.user_id                        WHERE u.email
= :email")        $this->db->bind(":email", $email)        return
$this->db->single()    }    func assignRole($userId, $roleName)
{        $this->db->query("SELECT id FROM roles WHERE name = :name")
$this->db->bind(":name", $roleName)        $role = $this->db->single()        if $role
{            $this->db->query("INSERT IGNORE INTO user_roles (user_id, role_id) VALUES
```

```
(:user_id, :role_id)")            $this->db->bind(":user_id", $userId)
$this->db->bind(":role_id", $role->id)            return $this->db->execute()        }
return false    }    func createWallet($userId) {        $this->db->query("INSERT INTO
wallets (user_id, balance, held_funds, currency, status)                          VALUES
(:user_id, 0.00, 0.00, 'NGN', 'active')")        $this->db->bind(":user_id", $userId)
return $this->db->execute()    }}
```

B8. Optional Library System — Plug and Play

> *Cache and JobQueue are NOT core Wolf libraries. They were designed by the Founder for the Selledge project specifically. Wolf makes it easy for any developer to build and use these patterns — but they are not generated by wolf new. They are created on demand and picked up automatically.*

B8.1 How Plug-and-Play Works

The Wolf compiler scans these directories at build time:

```
libraries/     # scanned firstmodels/        # scanned secondcontrollers/   # scanned
thirdservices/      # scanned fourthhelpers/       # scanned fifth
```

Any class file in any of these directories is automatically available everywhere in the project. No require statements. No registration. No bootstrap changes. The developer creates the file and uses the class immediately.

B8.2 wolf generate — Creating Optional Libraries

```
wolf generate library Cache      # creates libraries/Cache.wolf with starter template
wolf generate library JobQueue   # creates libraries/JobQueue.wolf with starter template
wolf generate library Mailer     # creates libraries/Mailer.wolf with starter template
wolf generate library Stripe     # creates libraries/Stripe.wolf with starter template
wolf generate model User         # creates models/User.wolf with starter templatewolf
generate controller Rides   # creates controllers/RidesController.wolfwolf generate
service Payment    # creates services/PaymentService.wolf
```

B8.3 Cache — Developer Builds It Their Way

wolf generate library Cache creates this starter. The developer fills in the logic their way — file-based like Selledge, or using Redis, or in-memory. Wolf does not care.

```
// libraries/Cache.wolf — generated starter// Fill this in however your project needs
caching.class Cache {    private $cacheDir    func __construct()
{        $this->cacheDir = APPROOT . "/cache"        make_dir($this->cacheDir)    }
func get($key) {        $file = $this->cacheDir . "/" . sha1($key) . ".cache"        if
file_exists($file) {            $data = unserialize(file_read($file))            if
time() < $data["expires"] { return $data["value"] }
file_delete($file)        }        return false    }    func set($key, $value, $ttl =
3600) {        $file = $this->cacheDir . "/" . sha1($key) . ".cache"        return
file_write($file, serialize([            "expires" => time() + $ttl,            "value"
```

```
=> $value,        ]))    }    func del($key)       { file_delete($this->cacheDir . "/" .
sha1($key) . ".cache") }    func exists($key)    { return $this->get($key) !== false }
func increment($key, $by = 1) {        $val = $this->get($key) ?? 0
$this->set($key, $val + $by)        return $val + $by    }}
```

B8.4 JobQueue — Developer Builds It Their Way

```
// libraries/JobQueue.wolf — generated starter// Mirrors the Selledge JobQueue.php
pattern exactly.class JobQueue {    private $db    private $queueName = "default"
const MAX_ATTEMPTS = 3    func __construct() {        $this->db = new Database    }
func push($jobType, $data) {        $payload = json_encode(["job" => $jobType, "data" =>
$data])        $this->db->query("INSERT INTO jobs (queue, payload, attempts,
available_at, created_at)                        VALUES (:queue, :payload,
0, :available_at, :created_at)")        $this->db->bind(":queue",
$this->queueName)        $this->db->bind(":payload",      $payload)
$this->db->bind(":available_at", time())        $this->db->bind(":created_at",   time())
return $this->db->execute()    }    func pop() {        $this->db->beginTransaction()
try {            $this->db->query("SELECT * FROM jobs WHERE queue = :queue
AND reserved_at IS NULL                                AND available_at <= :now
AND attempts < :max                                ORDER BY id ASC LIMIT 1 FOR UPDATE")
$this->db->bind(":queue", $this->queueName)            $this->db->bind(":now",   time())
$this->db->bind(":max",   self::MAX_ATTEMPTS)            $job = $this->db->single()
if $job {                $this->db->query("UPDATE jobs SET reserved_at = :now, attempts
= attempts + 1 WHERE id = :id")                $this->db->bind(":now", time())
$this->db->bind(":id",  $job->id)                $this->db->execute()            }
$this->db->commit()            if $job {                $payload =
json_decode($job->payload)                return ["id" => $job->id, "type" =>
$payload->job, "data" => $payload->data]            }            return nil        }
catch ($e) {            $this->db->rollBack()            log("JobQueue error:
{$e->message}")            return nil        }    }    func delete($jobId)
{        $this->db->query("DELETE FROM jobs WHERE id = :id")
$this->db->bind(":id", $jobId)        return $this->db->execute()    }    func
release($jobId, $delay = 60) {        $this->db->query("UPDATE jobs SET reserved_at =
NULL, available_at = :at WHERE id = :id")        $this->db->bind(":at", time() + $delay)
$this->db->bind(":id", $jobId)        return $this->db->execute()    }}
```

B9. Global Stdlib Functions — No Imports Ever

These functions are available everywhere in every Wolf file. No require. No use. No import. Exactly like PHP's built-in functions.

Function	Returns	PHP Equivalent
json_encode($data)	string	json_encode()
json_decode($json)	mixed	json_decode()
password_hash($pass)	string	password_hash($p, PASSWORD_BCRYPT)
password_verify($pass, $hash)	bool	password_verify()
jwt_encode($payload, $secret)	string	custom jwt_helper.php encode()

jwt_decode($token, $secret)	object|false	custom jwt_helper.php decode()
md5($str)	string	md5()
sha1($str)	string	sha1()
sha256($str)	string	hash('sha256', $str)
base64_encode($str)	string	base64_encode()
base64_decode($str)	string	base64_decode()
time()	int	time()
date($format)	string	date()
rand($min, $max)	int	rand()
round($n, $places=0)	float	round()
abs($n)	number	abs()
strlen($str)	int	strlen()
strtoupper($str)	string	strtoupper()
strtolower($str)	string	strtolower()
trim($str)	string	trim()
explode($sep, $str)	array	explode()
implode($sep, $arr)	string	implode()
str_contains($str, $sub)	bool	str_contains()
str_replace($find, $rep, $str)	string	str_replace()
array_push($arr, $val)	void	array_push()
array_pop($arr)	mixed	array_pop()
array_merge($a, $b)	array	array_merge()
count($arr)	int	count()
in_array($val, $arr)	bool	in_array()
serialize($val)	string	serialize()
unserialize($str)	mixed	unserialize()
file_exists($path)	bool	file_exists()
file_read($path)	string	file_get_contents()
file_write($path, $data)	bool	file_put_contents()
file_delete($path)	bool	unlink()
make_dir($path)	bool	mkdir($p, 0755, true)
scan_dir($path)	array	scandir()
app_root()	string	dirname(dirname(__FILE__))
log($msg)	void	error_log()
print($val)	void	echo / print

mail_send($opts)	bool	PHPMailer send
qr_generate($data, $fmt, $size)	string	QRCode_helper

B10. The @ml Block — Python ML Bridge

> *@ml blocks give Wolf developers access to the entire Python ML ecosystem without managing pip, venv, or Python versions. wolf.python handles the environment. The developer writes Python inside @ml and Wolf variables flow in and out automatically.*

```
// Basic @ml block@ml {    import numpy as np    result = np.array(items).mean()   #
items came from Wolf $items}print($result)   # available in Wolf after block exits//
Explicit in/out@ml(in: [$rideData, $zone], out: [$surgePrice, $confidence]) {    from
models import surge    surgePrice, confidence = surge.predict(rideData, zone)}// Async
@ml — does not block the HTTP response$task = @ml async {    from transformers import
pipeline    sentiment = pipeline("sentiment-analysis")(reviewText)}$result = await $task
```

B11. wolf.python — Python Environment

```
# wolf.python — committed to gitpython_version = "3.11"venv           = ".wolf-python"
# added to .gitignore automaticallyrequirements = [    "torch>=2.0",
"transformers>=4.35",    "scikit-learn>=1.3",    "numpy>=1.24",]
```

```
wolf python install        # install all from wolf.pythonwolf python add torch      #
add package + update wolf.pythonwolf python remove pandas  # remove packagewolf python
check          # verify all installedwolf python list           # show installed
versions
```

PART C — SPRINT TASK BOARDS

> *Tasks marked DONE from previous sprint versions are included for completeness. All new tasks start as TODO. A task is DONE only when acceptance criteria are met AND Founder has reviewed.*

C1. Month 1 — Compiler Foundation (Weeks 1–4)

Goal: Wolf source code compiles to a native binary via LLVM. Hello World runs. A basic HTTP endpoint responds to curl.

C1.1 Week 1 — Repository, Lexer & Tokens

Task	Owner	Week	Status	Acceptance Criteria
GitHub repo + Makefile	Founder	**W1**	**DONE**	Repo exists. Makefile has build/test/run. Go module initialised. cobra CLI skeleton: wolf --help works.
Define all token types	Claude	**W1**	**DONE**	token_types.go has every token from Part D spec. Every token typed. Reviewed by Gemini.
Implement Lexer	Claude	**W1**	**DONE**	Reads UTF-8. Emits []Token with type/literal/line/col/file. Handles $vars, {$interpolation}, @ml, # comments.
Lexer unit tests	Claude	**W1**	**DONE**	go test ./internal/lexer/... passes. Every token type covered. Edge cases: empty file, unicode, nested interpolation.
Error format standard	Claude	**W1**	**DONE**	All errors: wolf: error at line N, col N: <message>. Machine-parseable. Implemented once, reused everywhere.

C1.2 Week 2 — Parser & AST

Task	Owner	Week	Status	Acceptance Criteria
Define all AST node types	Claude	**W2**	**DONE**	ast_nodes.go: typed structs for every construct in Part B. All implement Node interface. Gemini reviewed.
Implement Parser	Claude	**W2**	**DONE**	Recursive-descent PEG. Correct precedence. Valid AST for all B2–B5 syntax. Helpful errors with line/col.
Parser unit tests	Claude	**W2**	**DONE**	Every AST node: 3+ passing tests, 1 error test. All Part B examples parse without error.
AST pretty-printer	Claude	**W2**	**DONE**	wolf fmt --ast source.wolf prints AST. Debug only.
Integration parse test	Claude	**W2**	**DONE**	Parse hello.wolf and a controller file without error. AST shape verified.

C1.3 Week 3 — LLVM IR Emitter (Replaces Go Emitter)

> *This is the most significant change from v3.x. We are not emitting Go. We are emitting LLVM IR directly. The llir/llvm Go package is used for programmatic IR construction.*

Task	Owner	Week	Status	Acceptance Criteria
Install + configure llir/llvm	Claude	**W3**	TODO	go get github.com/llir/llvm. Verify it compiles. Write a test that emits a trivial LLVM module and verifies its text output.
Implement Resolver	Claude	**W3**	TODO	Walk AST. Build scope tree. Strip $ prefix for LLVM output. Error on undeclared variable. Attach symbol table.
LLVM IR Emitter — variables	Claude	**W3**	TODO	Emit LLVM IR for: int, float, string, bool, nil, array, map assignments. Verify with llc.
LLVM IR Emitter — functions	Claude	**W3**	TODO	Emit LLVM IR for func declarations, params, return values, calls. Multiple return values as structs.
LLVM IR Emitter — control flow	Claude	**W3**	TODO	Emit LLVM IR for if/else, for, foreach, while, match. All compile to valid LLVM branch instructions.
wolf build + wolf run CLI	Claude	**W3**	TODO	wolf build file.wolf → native binary via llc + clang. wolf run file.wolf → compile to temp, execute. Both work end-to-end.
MILESTONE: Hello World	Founder	**W3**	TODO	print("Hello from Wolf!") compiles to native binary and runs. No Go involved. Founder demos. wolf run hello.wolf prints output.

C1.4 Week 4 — HTTP Server & Routing

Task	Owner	Week	Status	Acceptance Criteria
Wolf HTTP server — core	Claude	**W4**	TODO	Wolf runtime includes a native HTTP server. Handles GET/POST/PUT/DELETE. Routes registered at compile time. No Apache, no .htaccess.
Request object	Claude	**W4**	TODO	$req: ->body(), ->param($name), ->query($name), ->header($name), ->method(), ->json(). All tested.
Response object	Claude	**W4**	TODO	$res: ->json($data, $status=200), ->text($str), ->error($msg, $status), ->redirect($url), ->header($k,$v). All tested.
Core.wolf routing — URL parser	Claude	**W4**	TODO	Implement Router that parses /controller/method/param1/param2. Matches Core.php getUrl() exactly. camelCase conversion. Default controller=Home, method=index.
Controller auto-discovery	Claude	**W4**	TODO	Core.boot() scans controllers/ dir. Registers all *Controller.wolf classes. New controller file = new routes with zero other changes.
CORS middleware	Claude	**W4**	TODO	$core->cors([origins]) sets Access-Control headers correctly. OPTIONS preflight returns 200. Matches Selledge config.php CORS logic.
MILESTONE: HTTP API	Founder	**W4**	TODO	AuthController.wolf with login() and register() methods. GET /auth/login and POST /auth/register both respond to curl. Routing auto-discovered. No route file. Founder verifies.

C2. Month 2 — Datab[redacted]eks 5–8)

Goal: Full database stack working. Optional library system working. @ml with real Python ML working. Demo app end-to-end.

C2.1 Week 5 — Database Drivers & Library System

Task	Owner	Week	Status	Acceptance Criteria
config/config.wolf parser	Claude	**W5**	TODO	Wolf compiler reads config/config.wolf at startup. define() registers constants app-wide. Env::get() resolves from .env. Missing required key = clear error pointing to which key.
Wolf PDO wrapper — MySQL	Claude	**W5**	TODO	db_connect() stdlib function wraps Go database/sql + mysql driver. Returns handle with: prepare(), bindValue(), execute(), fetchAll(), fetch(), rowCount(), lastInsertId(), beginTransaction(), commit(), rollBack().
Database.wolf class compiles	Claude	**W5**	TODO	The full Database class from B7.2 compiles to LLVM IR and works correctly. Unit tested with real MySQL. query/bind/execute/single/resultSet/rowCount all produce correct results.
Redis built-in driver	Claude	**W5**	TODO	new Redis reads REDIS_HOST/PORT/PASS constants. Methods: get/set(ttl)/del/exists/expire/increment/hget/hset/lpush/lrange. Unit tested with real Redis instance.
MongoDB built-in driver	Claude	**W5**	TODO	new Mongo reads MONGO_HOST/PORT/DB constants. collection()->find/findOne/insertOne/updateOne/deleteOne. BSON↔Wolf map conversion correct.
Compiler class auto-discovery	Claude	**W5**	TODO	Compiler scans libraries/, models/, controllers/, services/, helpers/ at build time. Any class in these dirs is available everywhere without require. Zero runtime overhead.
wolf generate command	Claude	**W5**	TODO	wolf generate library Cache creates libraries/Cache.wolf with correct starter. wolf generate model User creates models/User.wolf. wolf generate controller Rides creates correct file. wolf generate service Payment creates services/PaymentService.wolf.

C2.2 Week 6 — @ml Bridge & wolf.python

Task	Owner	Week	Status	Acceptance Criteria
wolf.python parser	Claude	**W6**	TODO	Parse wolf.python config. Return WolfPythonConfig struct. Clear error on malformed config.
wolf python install/add/remove	Claude	**W6**	TODO	install: creates venv, runs pip, writes lock. add: installs + updates wolf.python + lock. remove: uninstalls + updates. All tested on Linux + macOS.
Auto venv activation	Claude	**W6**	TODO	wolf run detects @ml blocks. Checks wolf.python. Auto-

				creates venv. Auto-installs missing packages. Sets PYTHONPATH + LD_LIBRARY_PATH. End-to-end tested.
C bridge — wolf_runtime.c	Claude	**W6**	TODO	C file wraps CPython C API: Py_Initialize once, PyRun_String for @ml blocks, variable marshalling, Py_Finalize on exit. Compiles and links cleanly with LLVM output.
@ml lexer + parser	Claude	**W6**	TODO	AT_ML token. Python source inside @ml{} preserved as string literal — not lexed as Wolf. @ml(in:,out:) and @ml async variants parsed correctly into MLBlock AST node.
@ml LLVM IR emitter	Claude	**W6**	TODO	@ml block compiles to: (1) marshal in_vars to C structs, (2) call wolf_ml_execute() C function, (3) unmarshal out_vars from result. Generated IR is clean and readable.
GIL management	Claude	**W6**	TODO	Each @ml call acquires GIL via PyGILState_Ensure, releases via PyGILState_Release. 100 concurrent @ml goroutines do not deadlock. Stress test passes.
MILESTONE: PyTorch in Wolf	Founder	**W6**	TODO	Wolf file uses @ml to import torch, create tensor, run operation, return result to Wolf, print it. wolf run works. Founder verifies output matches expected.

C2.3 Week 7 — Full Model + Controller Stack

Task	Owner	Week	Status	Acceptance Criteria
User.wolf model complete	Claude	**W7**	TODO	Every method from B7.4 works: register, findUserByEmail, findUserByEmailOrPhone, findUserByEmailAndGetDetails, findUserById, assignRole, createWallet, updateLastLogin, updateProfile, updateUserPassword, getRolesForUser, getWallet.
Auth flow end-to-end	Claude	**W7**	TODO	POST /auth/register → User.register() → DB insert → wallet + role created → return userId. POST /auth/login → findUserByEmailOrPhone → password_verify → jwt_encode → return token. Both tested with curl.
Request security middleware	Claude	**W7**	TODO	Middleware validates requestID on POST/PUT/DELETE — mirrors Selledge config.php security check. Returns {status:false, message: Invalid Access} on missing/wrong requestID.
DB integration tests	Claude	**W7**	TODO	Docker Compose: MySQL + Redis + MongoDB. All CRUD operations tested against real databases. Transactions, rollback, row locking all work. CI runs these on every PR.
Services layer working	Claude	**W7**	TODO	Example PaymentService.wolf in services/ dir. Instantiated in controller. Auto-discovered by compiler. Shows services pattern works correctly.

C2.4 Week 8 — Showcase Demo App

Task	Owner	Week	Status	Acceptance Criteria
Design demo	Founder	**W8**	TODO	Ride-pricing API: wolf new surge-demo. Founder

architecture				approves schema, routes, and ML model before Claude builds.
wolf new surge-demo	Claude	**W8**	TODO	wolf new creates full structure. config/config.wolf filled. .env created. libraries/Database.wolf ready. wolf python check passes for scikit-learn.
Database layer	Claude	**W8**	TODO	Schema: drivers, ride_zones, rides tables. Seed data. RideModel queries drivers by zone using $this->db-> pattern.
@ml surge model	Claude	**W8**	TODO	scikit-learn LinearRegression trained on synthetic data. wolf.python manages package. @ml block loads model, predicts price multiplier.
HTTP layer	Claude	**W8**	TODO	POST /rides/estimate: validates requestID, queries DB, checks Redis cache, runs @ml, caches result 60s, returns {price, eta, confidence}. GET /health returns status.
MILESTONE: Full demo runs	Founder	**W8**	TODO	curl POST /rides/estimate returns correct JSON. Response consistently under 50ms. 1000 concurrent requests without crash or error. Founder records demo video — this is the launch asset.

C3. Month 3 — Polish, Docs & Launch (Weeks 9–12)

Goal: Wolf is publicly releasable. Documentation complete. All three platform binaries ship. Launch executed.

C3.1 Week 9 — Stdlib, Error Quality & wolf generate

Task	Owner	Week	Status	Acceptance Criteria
Complete stdlib functions	Claude	**W9**	TODO	All functions from B9 table implemented and tested. json_encode/decode, password_hash/verify, jwt_encode/decode, file_read/write/delete/exists, make_dir, sha1/sha256, date, time, rand, all string functions.
Error quality overhaul	Claude	**W9**	TODO	Every error shows: (1) source line, (2) caret pointing to exact position, (3) helpful message with suggestion. Model: Rust error quality. Zero raw panics on any user input. Ever.
wolf new — complete scaffolding	Claude	**W9**	TODO	wolf new myapp generates: config/config.wolf, libraries/Core.wolf, libraries/Controller.wolf, libraries/Database.wolf, server.wolf, .env.example, .gitignore, wolf.mod, README.md. wolf run server.wolf works immediately.
wolf generate — all types	Claude	**W9**	TODO	wolf generate library X, model X, controller X, service X all create correct starter files in correct directories. Generated files follow exact patterns from Part B.
wolf fmt	Claude	**W9**	TODO	wolf fmt source.wolf reformats Wolf source consistently. Idempotent. Handles all syntax from Part B.

C3.2 Week 10 — Documentation

Task	Owner	Week	Status	Acceptance Criteria
Getting Started	Claude	**W10**	TODO	docs/getting_started.md: install Wolf, Hello World, first HTTP controller, first DB query, first @ml block. Completable in 20 minutes. Every example tested and working.
Language Reference	Claude	**W10**	TODO	docs/syntax.md: every feature in Part B with working examples. The definitive Wolf syntax document.
Database Guide	Claude	**W10**	TODO	docs/database.md: config/config.wolf setup, .env, Database.wolf API, full model examples, Redis/MongoDB usage, transactions, Docker Compose for local dev.
Routing Guide	Claude	**W10**	TODO	docs/routing.md: URL pattern, controller conventions, camelCase conversion, request/response objects, middleware, CORS, requestID security pattern.
ML + Python Guide	Claude	**W10**	TODO	docs/ml_bridge.md + docs/python_env.md: @ml syntax, type bridge, wolf.python config, wolf python CLI, PyTorch/sklearn/HuggingFace examples, CI setup.
Optional Libraries Guide	Claude	**W10**	TODO	docs/libraries.md: what plug-and-play means, wolf generate usage, Cache and JobQueue starter patterns, how to build custom libraries.
README.md	Founder	**W10**	TODO	GitHub README. Wolf logo. One sentence: what it is. Code example (20 lines). Install instructions. Benchmarks teaser. Links to docs and demo video. Founder writes — it is the first impression.

C3.3 Week 11 — Platform Builds & CI

Task	Owner	Week	Status	Acceptance Criteria
Linux amd64 binary	Claude	**W11**	TODO	wolf binary for Linux x86-64. Self-contained — no Go, no LLVM, no Python needed for non-@ml code. Tested on Ubuntu 22.04. Released as wolf-linux-amd64.
macOS arm64 binary	Claude	**W11**	TODO	wolf binary for macOS Apple Silicon. Released as wolf-darwin-arm64. Tested on M2 Mac.
macOS amd64 binary	Claude	**W11**	TODO	wolf binary for macOS Intel. Released as wolf-darwin-amd64.
GitHub Actions CI	Claude	**W11**	TODO	On PR: unit tests, integration tests (Docker), example compilation. PR blocked if any fail. On merge to main: build all 3 platform binaries, publish to GitHub Releases automatically.
End-to-end test suite	Claude	**W11**	TODO	20 Wolf programs covering all features. Pass on all 3 platforms: variables, functions, classes, HTTP routing, DB queries, Redis, @ml, concurrency, error handling, optional libraries.
Benchmark suite	Claude	**W11**	TODO	Measure: requests/sec vs Go equivalent. Latency percentiles under load. GC pause comparison (Wolf has none). Publish results in docs/benchmarks.md.

C3.4 Week 12 — Launch

Task	Owner	Week	Status	Acceptance Criteria
GitHub repo public	Founder	**W12**	TODO	Repo public. Apache 2.0 license. CONTRIBUTING.md. Good First Issue label on 5 tickets.
GitHub Releases v0.1.0	Claude	**W12**	TODO	Tag v0.1.0. All 3 platform binaries attached. Changelog written. Release notes honest about what works and what is Phase 2.
Hacker News Show HN	Founder	**W12**	TODO	Post Tuesday 09:00 ET: Show HN: Wolf — PHP syntax, LLVM speed, Python ML, zero config routing. Link to demo video. Link to GitHub.
Dev.to launch article	Founder	**W12**	TODO	Why I built a compiled language with Claude and Gemini. Personal story, Selledge origin, code examples, performance numbers. Published same day as HN.
Demo video	Founder	**W12**	TODO	3–5 minutes. Install Wolf, wolf new myapp, write controller + model, wolf run server.wolf, curl the API, show response time. Upload to YouTube. Link in README.
Reply to HN comments	Founder	**W12**	TODO	Reply to every HN comment on launch day. Traction comes from conversation. Most important Founder task on launch day.

PART D — CODE SPECIFICATION

D1. Complete Token List

Token	Example	Notes
DOLLAR_IDENT	$myVar	$ + [a-zA-Z_][a-zA-Z0-9_]*
STRING	"hello"	Double-quoted. Contains interpolation segments.
STRING_INTERP	"hello {$name}"	Emitted as: START, segments, END
INT	42, -7	64-bit signed integer
FLOAT	3.14, -0.5	64-bit float
BOOL	true, false	Lowercase only
NIL	nil	Null value
FUNC	func	Function keyword
CLASS	class	Class definition
EXTENDS	extends	Inheritance
NEW	new	Object instantiation
CONST	const	Constant declaration inside class
SELF	self	Class self reference (inside class only)
IF / ELSE	if, else	Conditional
FOR	for	C-style for loop
FOREACH / AS	foreach, as	Iteration
WHILE	while	While loop
MATCH	match	Pattern match / exhaustive switch
RETURN	return	Return statement
ASYNC	async	Async task
AWAIT	await	Await result
PARALLEL	parallel	Parallel block
AT_ML	@ml	ML block
TRY / CATCH	try, catch	Error handling
VAR	var	Typed variable declaration
STRICT	strict	Enable strict mode for this file
DEFINE	define	Constant definition
ARROW	=>	Match arm / arrow function body
FAT_ARROW	->	Property / method access
DOUBLE_COLON	::	parent:: only — banned in all other contexts
LBRACE/RBRACE	{ }	Block delimiters

LPAREN/RPAREN	( )	Grouping / params
LBRACKET/RBRACKET	[ ]	Array access / literals
COMMA	,	Separator
COLON	:	Map key, type annotation, named param
HASH	#	Comment — rest of line ignored
DOT	.	String concatenation
NULLCOAL	??	Null coalescing
PLUS/MINUS/STAR/SLASH/PERCENT	+ - * / %	Arithmetic
EQ/NEQ/LT/GT/LTE/GTE	== != < > <= >=	Comparison
AND/OR/NOT	&& || !	Logical
ASSIGN	=	Assignment
COMPOUND_ASSIGN	+= -= *= /= .=	Compound assignment
INCR/DECR	++ --	Increment / decrement
EOF	(end of file)	Signals end of token stream

D2. Core Go Structs & Interfaces

D2.1 Token

```
// internal/lexer/token.gotype TokenType stringtype Token struct {    Type    TokenType
Literal string    Line    int    Col     int    File    string}
```

D2.2 AST Node Types

```
// internal/parser/ast_nodes.gotype Node     interface { nodeType() string; Pos()
Position }type Stmt     interface { Node; stmtNode() }type Expr     interface { Node;
exprNode() }type Position struct    { Line, Col int; File string }type ProgramNode
struct { Stmts []Stmt; Pos_ Position }type VarAssignStmt  struct { Name string; Value
Expr; TypeAnn string; Pos_ Position }type DefineStmt     struct { Name string; Value
Expr; Pos_ Position }type FuncDecl       struct { Name string; Params []Param;
ReturnTypes []string; Body []Stmt; IsArrow bool; Pos_ Position }type ClassDecl
struct { Name string; Parent string; Fields []Field; Methods []FuncDecl; Consts
[]ConstDecl; Pos_ Position }type IfStmt         struct { Cond Expr; Then []Stmt; ElseIf
[]ElseIfClause; Else []Stmt; Pos_ Position }type ForStmt        struct { Init Stmt; Cond
Expr; Post Stmt; Body []Stmt; Pos_ Position }type ForEachStmt    struct { Key, Value
string; Iter Expr; Body []Stmt; Pos_ Position }type WhileStmt      struct { Cond Expr;
Body []Stmt; Pos_ Position }type MatchStmt      struct { Subject Expr; Arms []MatchArm;
Pos_ Position }type TryCatch       struct { Try []Stmt; ErrVar string; Catch []Stmt;
Pos_ Position }type AsyncBlock     struct { Body []Stmt; IsAwait bool; Pos_ Position }
type ParallelBlock  struct { Assignments []VarAssignStmt; Pos_ Position }type ReturnStmt
struct { Value Expr; Pos_ Position }type MLBlock        struct { PythonSrc string;
InVars []string; OutVars []string; ModelName string; IsAsync bool; Pos_ Position }type
StrictDecl     struct { Pos_ Position }
```

D2.3 Wolf Runtime — Database Driver Contract

```
// internal/runtime/database.go// WolfDB backs the Database class in
libraries/Database.wolf.type WolfDB struct {    db    *sql.DB    stmt  *sql.Stmt    err
error}func NewWolfDB(dsn, user, pass string, opts map[string]any) (*WolfDB, error)func
(w *WolfDB) Query(sql string) errorfunc (w *WolfDB) Bind(param string, value any) error
func (w *WolfDB) Execute() boolfunc (w *WolfDB) ResultSet() []map[string]anyfunc (w
*WolfDB) Single() map[string]anyfunc (w *WolfDB) RowCount() intfunc (w *WolfDB)
LastInsertId() int64func (w *WolfDB) BeginTransaction() errorfunc (w *WolfDB) Commit()
errorfunc (w *WolfDB) RollBack() error// WolfRedis backs new Redis in Wolf models.type
WolfRedis struct { client *redis.Client }func NewWolfRedis(host, port, pass string)
(*WolfRedis, error)func (r *WolfRedis) Get(key string) (string, error)func (r
*WolfRedis) Set(key string, value any, ttl int) errorfunc (r *WolfRedis) Del(key string)
errorfunc (r *WolfRedis) Exists(key string) boolfunc (r *WolfRedis) Expire(key string,
ttl int) errorfunc (r *WolfRedis) HGet(key, field string) (string, error)func (r
*WolfRedis) HSet(key, field string, value any) errorfunc (r *WolfRedis) LPush(key
string, values ...any) errorfunc (r *WolfRedis) LRange(key string, start, stop int64)
([]string, error)func (r *WolfRedis) Increment(key string, by int64) (int64, error)//
WolfMongo backs new Mongo in Wolf models.type WolfMongo struct { client *mongo.Client;
db *mongo.Database }func NewWolfMongo(host, port, dbname string) (*WolfMongo, error)func
(m *WolfMongo) Collection(name string) *WolfCollection
```

D2.4 ML Bridge Contract

```
// internal/mlbridge/bridge.gotype MLBridge interface {    Init(pythonPath string) error
Execute(src string, inVars map[string]any) (map[string]any, error)    ExecuteAsync(src
string, inVars map[string]any) <-chan MLResult    LoadModel(name string, loadScript
string) error    Shutdown()}type MLResult struct { Vars map[string]any; Err error }
```

D2.5 wolf.python Config Structs

```
// internal/pythonenv/config.gotype WolfPythonConfig struct {    PythonVersion    string
VenvPath         string    Requirements     []string    RequirementsFile string
ExtraIndexURL    string    InstallFlags     []string}type LockedPackage struct { Name,
Version, Hash string }type WolfPythonLock struct {    GeneratedAt   time.Time
PythonVersion string    Packages      []LockedPackage}
```

PART E — TESTING & LAUNCH

E1. Testing Strategy

Unit tests	Every function in every package. 90%+ coverage target. Go testing + testify. Run on every commit.
Integration tests	Compile and run real Wolf programs end-to-end. 20+ programs. Docker for DB tests. Run on every PR.
Performance tests	Requests/sec vs Go equivalent. Latency percentiles. GC pause comparison. Regression blocks merge.
Stress tests	1000 concurrent HTTP requests. 100 concurrent @ml goroutines. DB pool exhaustion. All must pass.
Negative tests	50+ invalid Wolf programs. All produce helpful errors with line/col and suggestions. Zero panics ever.
Pattern tests	Verify :: never appears in any generated or library code except parent::. Enforce no DB:: anywhere.

E2. The Five Mandatory Demos

> *All five must pass on all three platforms before the repo goes public. These are the project acceptance tests.*

1. **Demo 1** Hello World: wolf run hello.wolf prints Hello from Wolf! — native binary, no Go, no runtime.
2. **Demo 2** HTTP API: wolf new myapp → write AuthController with login/register → wolf run server.wolf → curl POST /auth/register returns userId, POST /auth/login returns JWT token.
3. **Demo 3** Database: User model with register/findUserByEmail/assignRole/createWallet all hitting real MySQL via $this->db-> pattern. Redis session set/get working via new Redis.
4. **Demo 4** Concurrency: 1000 concurrent POST /auth/login requests. All return correct response. No crashes. P99 latency under 20ms.
5. **Demo 5** Full Stack ML: surge-demo — POST /rides/estimate hits MySQL for driver data, checks Redis cache, runs @ml scikit-learn model, returns {price, eta, confidence}. curl verified.

E3. Launch Checklist

E3.1 Pre-Launch

- All 5 mandatory demos pass on Linux amd64, macOS arm64, macOS amd64
- README.md written and approved by Founder
- Getting Started guide tested end-to-end by someone reading it fresh
- GitHub Actions CI green on main branch
- All 3 platform binaries attached to v0.1.0 release
- Zero panics on any invalid Wolf input — fuzzing tested
- Apache 2.0 LICENSE added
- CONTRIBUTING.md written
- 5 example projects in examples/ directory
- .env.example committed with all keys documented
- No :: anywhere except parent:: — verified by test

E3.2 Launch Day Sequence

08:00	Publish GitHub Releases v0.1.0 — binaries live
08:05	Make repository public
08:30	Publish Dev.to article
09:00 ET — Tuesday only	Post Hacker News Show HN
09:30	Post to r/programming r/golang r/PHP r/Python
10:00	Share demo video everywhere
All day — Founder priority	Reply to every HN comment

E4. Phase 2 — After Launch

If launch generates traction the Founder decides order based on community feedback.

- **Priority 1:** wolfpkg — Wolf package manager and registry. First thing the community will ask for.
- **Priority 2:** WolfMVC — Full MVC framework. Models with migrations, views with templating, routing conventions, validation, ORM. This is what brings PHP developers at scale.
- **Priority 3:** WebAssembly target — Wolf compiles to WASM for browser execution. LLVM's WASM backend makes this achievable. Frontend Wolf.
- **Priority 4:** VS Code extension — syntax highlighting and language server. Developers request this immediately after any language launch.
- **Priority 5:** Windows support — wolf binary for Windows amd64.
- **Priority 6:** Benchmarks published — official Wolf vs Go vs PHP vs Python numbers. Drives press coverage.
- **Priority 7 — The dream:** Wolf becomes self-hosted — Wolf compiler rewritten in Wolf. The language bootstraps itself. The milestone every serious language hits.

End of Wo he Real Build

Now rt building.